单调栈

739.每日温度

给定一个整数数组 `temperatures` ，表示每天的温度，返回一个数组 `answer` ，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

示例 1:

输入: temperatures = [73,74,75,71,69,72,76,73]
输出: [1,1,4,2,1,1,0,0]

代码块

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        #单调栈：从后往前遍历，是为了在处理当前元素时，右边的信息已经准备好，从而用单调栈直接得到答案。
        stack =[]
        n =len(temperatures)
        ans =[0]*n
        for i in range(n-1,-1,-1):  #从后向前遍历是为了先将右侧的值展现出来。
            while stack and temperatures[stack[-1]]<=temperatures[i]:
                stack.pop()
                #保持栈是单调递减的
            if stack:
                ans[i] = stack[-1]-i
            stack.append(i)
        return ans
```

496.下一个更大的元素I

`nums1` 中数字 `x` 的 **下一个更大元素** 是指 `x` 在 `nums2` 中对应位置 **右侧** 的 **第一个** 比 `x` 大的元素。

给你两个 **没有重复元素** 的数组 `nums1` 和 `nums2` ，下标从 **0** 开始计数，其中 `nums1` 是 `nums2` 的子集。

对于每个 `0 <= i < nums1.length` ，找出满足 `nums1[i] == nums2[j]` 的下标 `j` ，并且在 `nums2` 确定 `nums2[j]` 的 **下一个更大元素** 。如果不存在下一个更大元素，那么本次查询的答案是 `-1` 。

返回一个长度为 `nums1.length` 的数组 `ans` 作为答案，满足 `ans[i]` 是如上所述的 **下一个更大元素** 。

示例 1：

输入： nums1 = [4,1,2], nums2 = [1,3,4,2].
输出： [-1,3,-1]
解释： nums1 中每个值的下一个更大元素如下所述：
nums2 = [1,3,**4**,2]。不存在下一个更大元素，所以答案是 -1 。
nums2 = [***1***,3,4,2]。下一个更大元素是 3 。
nums2 = [1,3,4,***2***]。不存在下一个更大元素，所以答案是 -1 。

代码块

```
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
#方法1
#暴力
        m,n =len(nums1),len(nums2)
        res =[-1]*m
        for i in range(m):
            j =nums2.index(nums1[i])
            k = j+1
            while k<n and nums2[k]<nums2[j]:
                k+=1
            res[i] = nums2[k] if k<n else -1
        return res
#方法2:
#利用单调栈的方法，提前计算nums2中每个元素的下一个最大元素
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        #哈希表 + 单调栈
        #从左到右遍历，记录nums2中每个元素的下一个更大值
        idx ={x:i for i ,x in enumerate(nums1)}
        ans =[-1]*len(nums1)
        st =[]
        for x in nums2:
            while st and x>st[-1]:
                ans[idx[st.pop()]] =x
            if x in idx:
                st.append(x)  #只需把num1中的元素入栈
        return ans
```

503.下一个更大的元素II

给定一个**循环数组** `nums` （ `nums[nums.length - 1]` 的下一个元素是 `nums[0]` ），返回 *`nums` 中每个元素的* ***下一个更大元素***。

数字 `x` 的 **下一个更大的元素** 是按数组遍历顺序，这个数字之后的第一个比它更大的数，这意味着你应该循环地搜索它的下一个更大的数。如果不存在，则输出 `-1` 。

示例 1:

输入: nums = [1,2,1]
输出: [2,-1,2]
解释: 第一个 1 的下一个更大的数是 2；
数字 2 找不到下一个更大的数；
第二个 1 的下一个最大的数需要循环搜索，结果也是 2。

本题 nums 是一个循环数组，nums[n－1] 右边是 nums[0]。我们可以把 nums 复制一份，拼在 nums 右边，这样就把环形数组变成一般数组了。例如 [1,2,1] 变成 [1,2,1,1,2,1]。

本题要计算每个元素的下一个更大元素的值（注意是值不是下标），这是单调栈的标准应用。

代码块

```
class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans =[-1]*n
        stack =[]
        for i in range(2*n-1,-1,-1):
            while stack and nums[i%n]>=stack[-1]:
                stack.pop()
            if stack and i<n:
                ans[i] = stack[-1]
            stack.append(nums[i%n])
        return ans
```

42.接雨水

给定 `n` 个非负整数表示每个宽度为 `1` 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1：

> **输入：** height = [0,1,0,2,1,0,1,3,2,1,2,1]
> **输出：** 6
> **解释：** 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

代码块

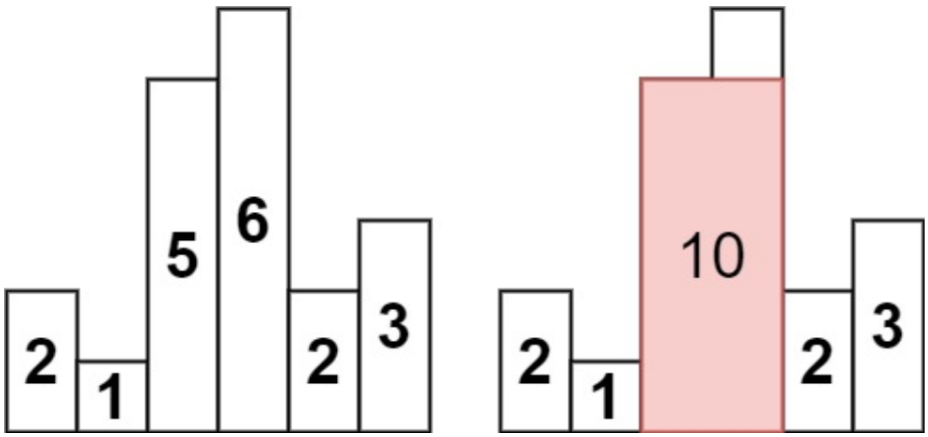
```
class Solution:
    def trap(self, height: List[int]) -> int:
        ans =0
        n =len(height)
        prenum =[0]*n # pre_max[i] 表示从 height[0] 到 height[i] 的最大值
        prenum[0] = height[0]
        for i in range(1,n):
            prenum[i] = max(prenum[i-1],height[i])
        sufnum =[0]*n# suf_max[i] 表示从 height[i] 到 height[n-1] 的最大值
        sufnum[n-1] =height[n-1]
        for j in range(n-2,-1,-1):
            sufnum[j] = max(sufnum[j+1],height[j])
        for h,pre,suf in zip(height,prenum,sufnum):
            ans +=min(pre,suf)-h
        return ans
```

84.柱状图中的最大矩形

给定 *n* 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1 。

求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例 1:



> **输入:** heights = [2,1,5,6,2,3]
> **输出:** 10
> **解释:** 最大的矩形为图中红色区域，面积为 10

最终矩形的最大面积一定是以heights数组中的元素为高的，只需要枚举每个高度heights[i]，不断的向左右两侧扩展宽度，从而找到最大宽度，通过这种方法可以找到所有高度的最大宽度，从而就可以算出最大的矩形面积。

时间复杂度：如果通过暴力的方法，通过两层for循环找到每个高度的最大左右边界，时间复杂度=O(n^2).

有什么方法不用通过两层for循环就能找到左右边界呢?

单调栈！！！

从左向右遍历，如果当前元素比栈顶元素大，就不断的出栈，直到找比当前元素小的索引下标，就这样不断地找到每个高度左边比它小的那一个索引位置left[i];

从右向左遍历，如果当前元素比栈顶元素大，就不断的出栈，直到找比当前元素小的索引下标，就这样不断地找到每个高度右边比它小的那一个索引位置right[i];

这样通过一次for循环就可以找到所有高度的left[i],right[i]。

代码块

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n =len(heights)
        ans =0
        left =[-1]*n
        stack1 =[]
        #每个元素都要出栈
        #这里每个元素只入栈一次，出栈一次，所以时间复杂度为O(n)
        for i in range(n):
            while stack1 and heights[stack1[-1]]>=heights[i]:
                stack1.pop()
```

```
            if stack1:
                left[i] = stack1[-1]
            stack1.append(i)
        right =[n]*n
        stack2 =[]
        for j in range(n-1,-1,-1):
            while stack2 and heights[stack2[-1]]>=heights[j]:
                stack2.pop()
            if stack2:
                right[j] = stack2[-1]
            stack2.append(j)
        for h,l,r in zip(heights,left,right):
            ans =max(ans,(r-l-1)*h)
        return ans
```

215.数组中的第K个最大元素

给定整数数组 `nums` 和整数 `k`，请返回数组中第 **`k`** 个最大的元素。

请注意，你需要找的是数组排序后的第 `k` 个最大的元素，而不是第 `k` 个不同的元素。

你必须设计并实现时间复杂度为 `O(n)` 的算法解决此问题。

代码块

```
输入: [3,2,1,5,6,4], k = 2
输出: 5
```

代码块

```
class Solution:
    #快速排序时间复杂度：O(nlogn)
    #快速排序每次都能确定一个元素的最终位置（基准）
    #快速选择排序时间复杂度：O(n)
    #设 N 为数组长度。根据快速排序原理，如果某次哨兵划分后，基准数的索引正好是 N-k ，则意味着它就是第 k 大的数字 。
    def findKthLargest(self, nums: List[int], k: int) -> int:
        target = len(nums)-k
        def quick(left:int,right:int)->int:
            pivot_index = random.randint(left,right)
            nums[left],nums[pivot_index] = = nums[pivot_index], nums[left]
            pivot = nums[left]
            i, j = left, right
            while i<j:
                while i<j and nums[j]>=pivot:
                    j-=1
                while i<j and nums[i]<=pivot:
```

```
                    i+=1
                if i<j:
                    nums[i],nums[j] = nums[j],nums[i]
            nums[left],nums[i] = nums[i],nums[left]
            return i #这就是piovt基准元素所在地 ，是第i+1个最小值
        left, right = 0, len(nums) - 1
        while left <= right:
            pivot_pos = partition(left, right)

            if pivot_pos == target:
                return nums[pivot_pos]
            elif pivot_pos < target:
                left = pivot_pos + 1
            else:
                right = pivot_pos - 1

        return -1
```

901.股票价格跨度

设计一个算法收集某些股票的每日报价，并返回该股票当日价格的 **跨度** 。

当日股票价格的 **跨度** 被定义为股票价格小于或等于今天价格的最大连续日数（从今天开始往回数，包括今天）。

- 例如，如果未来 7 天股票的价格是 `[100,80,60,70,60,75,85]` ，那么股票跨度将是 `[1,1,1,2,1,4,6]` 。

实现 `StockSpanner` 类：

- `StockSpanner()` 初始化类对象。
- `int next(int price)` 给出今天的股价 `price` ，返回该股票当日价格的 **跨度** 。

示例：

> **输入**：
> ["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
> [[], [100], [80], [60], [70], [60], [75], [85]]
> **输出**：
> [null, 1, 1, 1, 2, 1, 4, 6]
>
> **解释：**
> StockSpanner stockSpanner = new StockSpanner();
> stockSpanner.next(100); // 返回 1
> stockSpanner.next(80); // 返回 1
> stockSpanner.next(60); // 返回 1
> stockSpanner.next(70); // 返回 2
> stockSpanner.next(60); // 返回 1
> stockSpanner.next(75); // 返回 4 ，因为截至今天的最后 4 个股价 (包括今天的股价 75) 都小于或等于今天的股价。
> stockSpanner.next(85); // 返回 6

代码块

```python
class StockSpanner:

    def __init__(self):
        self.stack =[]
        self.cur_day = -1

    def next(self, price: int) -> int:
        while self.stack and self.stack[-1][1]<=price:
            self.stack.pop()
        self.cur_day +=1
        if self.stack:
            last_day = self.stack[-1][0]
        else:
            last_day =-1
        self.stack.append((self.cur_day,price))
        return self.cur_day -last_day
```

402.移除K位数字

给你一个以字符串表示的非负整数 `num` 和一个整数 `k` ，移除这个数中的 `k` 位数字，使得剩下的数字最小。请你以字符串形式返回这个最小的数字。

示例 1：

输入： num = "1432219", k = 3
输出： "1219"
解释： 移除掉三个数字 4, 3, 和 2 形成一个新的最小的数字 1219 。

代码块

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        stack =[]
        #从左向右遍历，先将左边的大元素删除
        for digit in num:
            while k and stack and digit<stack[-1]:
                stack.pop()
                k-=1
            stack.append(digit)
        if k:    #已经是单调递增的了，右边的元素大，可以直接去掉
            ans = stack[:-k]
        else:
            ans = stack
        return ''.join(ans).lstrip('0') or "0"
        #如果前面的结果是空字符串 ""，Python 会把它当成 False，于是返回后面的 "0"
```

581.最短无序连续子数组

给你一个整数数组 `nums` ，你需要找出一个 **连续子数组** ，如果对这个子数组进行升序排序，那么整个数组都会变为升序排序。

请你找出符合题意的 **最短** 子数组，并输出它的长度。

代码块

```
输入：nums = [2,6,4,8,10,9,15]
输出：5
解释：你只需要对 [6, 4, 8, 10, 9] 进行升序排序，那么整个表都会变为升序排序。
```

代码块

```
class Solution:
    def findUnsortedSubarray(self, nums: List[int]) -> int:
        #从左向右遍历，不断的记录最大值，如果后面有比最大值小的元素，说明这个元素是要参与到排序中的
        #从右向左遍历，不断记录最小值，如果右边有比最小值大的元素，说明这个元素是要参与到排序中的
        n =len(nums)
        min =nums[n-1]
        max =nums[0]
        begin,end =0,-1
        for i in range(n):
            if nums[i]<max:
                end =i
```

```
            else:
                max =nums[i]
            if nums[n-i-1]>min:
                begin = n-i-1
            else:
                min =nums[n-i-1]
        return end-begin+1

```